

CAD-ANALYZER

Fichas de Extração — VIGAS

■ Esta ficha responde: dado um texto/linha do DXF, qual campo JSON extrair e como.

#	Seção	Conteúdo
1	Identificação	RE_VIGA · padrões · balanço BA*/VB*
2	Geometria LV vs FV	LINE entities · diagrama corte + planta
3	Balanço	BA*/VB* → apoio_fim="" + diagrama
4	Dimensões b/h	RE_DIM · RE_DIM_BH · convenção b menor que h
5	Schema JSON Completo	FichaFase3Viga · tramos · apoios
6	Layers Canônicos	fundo · Escoras · GARFOS · aliases
7	Confidence e Fallbacks	fórmula · cadeia de 6 níveis
8	Matriz de Decisão	balanço · sem dimensão · layer desconhecido
9	Exemplos Reais	V101 · BA-5 (balanço)
10	Pipeline Completo	fluxo E2E · checklist 12 passos

1

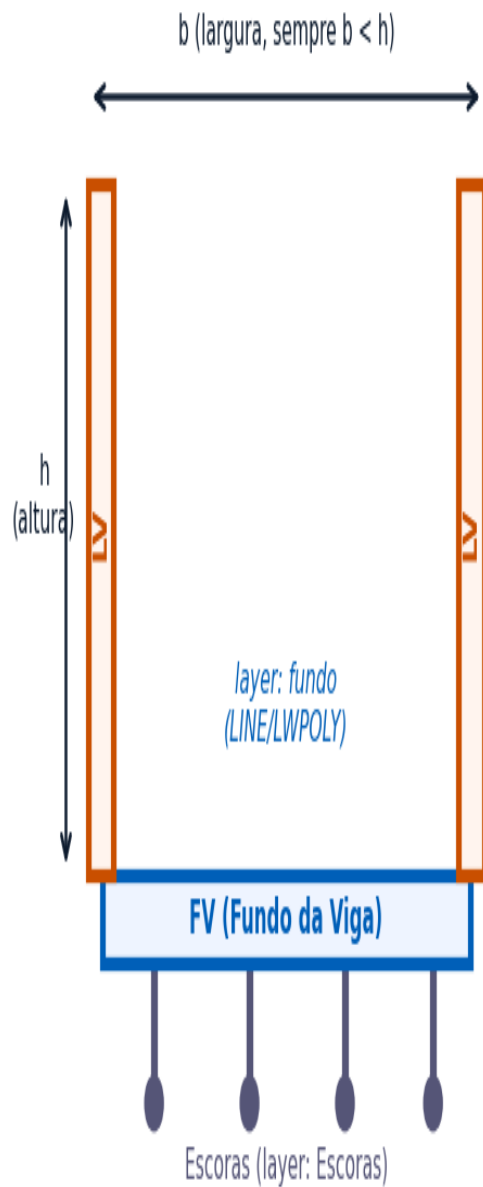
Identificação — RE_VIGA

```
RE_VIGA = re.compile(
    r'^ (V|BA|VB|VT|VC) \. ? - ? \d + ( [A-Z] | \. / \d + ) ? ',
    re.IGNORECASE
)
# V=normal BA*=balanço VB*=borda VT*=topo VC*=coroamento
```

Texto DXF	Casou?	Tipo	Obs
V101	✓	Normal	2 apoios esperados
V101A	✓	Normal	Sufixo letra
BA - 5	✓	Balanço	apoio_fim="" é CORRETO
VB3	✓	Borda balanço	apoio_fim="" é CORRETO
VT - 2	✓	Topo	Travamento
VC1	✓	Coroamento	—
L5	✗	—	É laje
P3	✗	—	É pilar

■ LV = painéis laterais (layer Painéis). FV = prancha de fundo (layer fundo). Distinção CRÍTICA para cálculo de material.

Componentes da Viga (Vista Frontal)



Vista Superior – Apoios e Comprimento

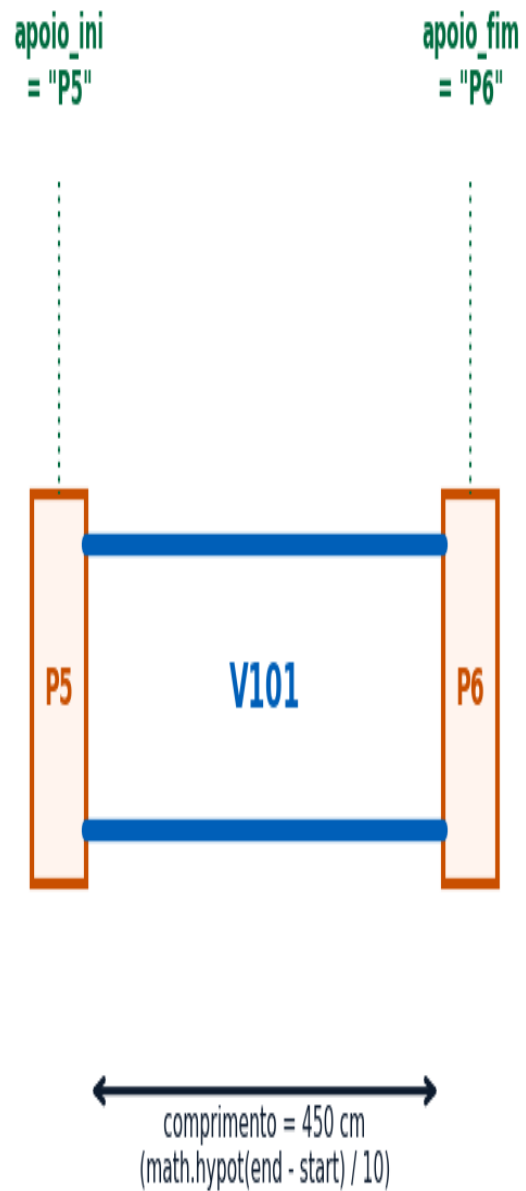


Figura 1 — Corte transversal da viga (LV/FV/Escoras) e vista superior com apoios

```
# Acesso às LINEs de fundo (FV):
for e in msp.query('LINE'):
    if normalize_layer(e.dxf.layer) not in ('FUNDO', 'FUNDOS'): continue
    start = (float(e.dxf.start.x), float(e.dxf.start.y))
    end   = (float(e.dxf.end.x),   float(e.dxf.end.y))
    comp_cm = math.hypot(end[0]-start[0], end[1]-start[1]) / 10
    fv_candidates.append({'start':start, 'end':end, 'comp':comp_cm})
```

Viga Normal vs Balanço (BA*/VB*)

Viga Normal V101

Viga Balanço BA-5

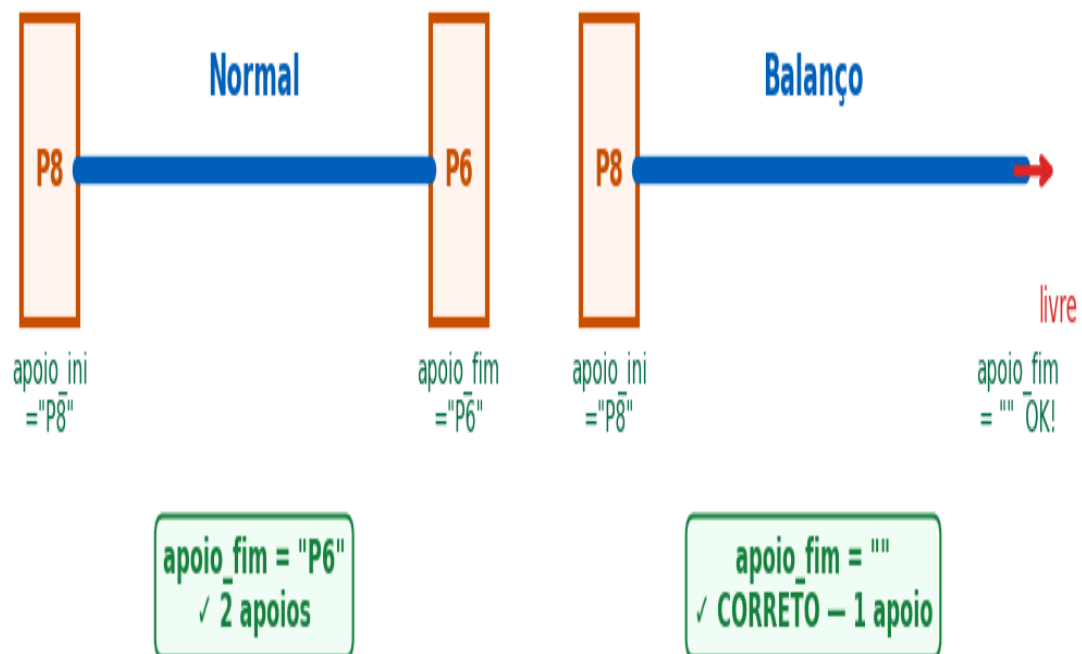


Figura 2 — Viga normal (2 apoios) vs viga em balanço BA*/VB* (1 apoio, apoio_fim="")

✓ BA* ou VB* com apoio_fim="" NÃO é erro. É o comportamento correto e esperado.

```
# Lógica de detecção:
is_balanco = bool(re.match(r'^(BA|VB)', codigo, re.I))

tramo = {
    'apoio_ini': 'P8',    # sempre preenchido
    'apoio_fim': '',      # '' se é balanço – CORRETO
    'comprimento': 220.0
}
# is_balanco=True → NÃO penalizar confidence por apoio_fim vazio
```

```

RE_DIM    = re.compile(r'(\d{1,3})\s*[xX*\/]\s*(\d{1,3})')
RE_DIM_BH = re.compile(r'b\s*=\s*(\d{1,3}).*?h\s*=\s*(\d{1,3})', re.I)

# CONVENÇÃO: b (largura) < h (altura)
largura = min(a, b) # cm - b
altura  = max(a, b) # cm - h

```

Texto DXF	largura b	altura h	Obs
"20x50"	20 cm	50 cm	b < h ✓
"50x20"	20 cm	50 cm	Reordenado: b=min, h=max
"b=20 h=50"	20 cm	50 cm	RE_DIM_BH

■ Sem dimensão em VIGA_SEARCH_RADIUS=1200mm: largura=0, altura=0, confidence == 0.30


```
{
  "codigo": "V101",
  "pavimento": "1_PAVIMENTO",  "obra_nome": "ALIMONTI",

  "largura": 20.0,                # cm – b
  "altura": 50.0,                 # cm – h

  "tramos": [{
    "apoio_ini": "P5",            # pilar inicial (extremidade start)
    "apoio_fim": "P6",           # pilar final (" se BA*/VB*)
    "comprimento": 450.0,        # cm
    "nivel": 2.80                # m
  }],

  "fv_segs": [{"start": [15000, 10000], "end": [19500, 10000]}],

  "escoras": 12,                 # COUNT LINES layer Escoras
  "garfos": 6,                   # COUNT INSERTs layer GARFOS

  "confidence": 0.90,  "revisado": false
}
```

Campo	Origem DXF
codigo	TEXT/MTEXT → RE_VIGA
largura / altura	RE_DIM/RE_DIM_BH ≤ 1200mm → b=min, h=max
tramos.apoio_ini/fim	Extremidades da LINE de fundo → pilar mais próximo ≤ 1200mm
tramos.comprimento	math.hypot(end – start) em mm → / 10 para cm
fv_segs	LINE layer fundo: start + end em mm (coordenadas DXF)
escoras	COUNT(LINE layer Escoras) próximas à viga
garfos	COUNT(INSERT layer GARFOS) próximos à viga

Canônico	Aliases reais	Entidade	Uso
BEAM_BOTTOM	fundo, FUNDOS, "Fundo da Viga"	LINE/LWPOLY	FV — comprimento da viga
SHORING	Escoras, "Escora de Viga"	LINE/INSERT	Apoio vertical
FORK_METAL	GARFOS	INSERT/LINE	Garfos HT20CT
CLAMP_METAL	presilha, Presilha, PRESILHA	LWPOLY/LINE	Presilhas metálicas
ANCHOR_BAR_LV	"BARRA DE ANCORAGEM"	LINE	Barras (≠ PL: "BARRA ANCORAGEM")
SPACER	Forcador	INSERT/LINE	Espaçadores
BATTEN_BEAM	barrote	LINE/LWPOLY	Barrotes da viga
ELEMENT_LABEL	NOMENCLATURA, texto	TEXT/MTEXT	Textos ID (V101)

Nível	Condição	Confidence
1	Texto em NOMENCLATURA → LINE em fundo/Painéis $\leq 1200\text{mm}$	raio_score
2	Texto → LINE em qualquer layer (não fundo/Painéis)	raio_score – 0.15
3	Texto + RE_DIM → dimensões extraídas	+0 (esperado)
4	Texto + RE_DIM_BH (b=20 h=50)	+0 (aceito)
5	Texto sem dimensão $\leq 1200\text{mm}$	conf – 0.30
6	BA*/VB* com apoio_fim=""	OK — balanço não penaliza

Situação	Ação	Confidence
Viga BA*/VB* (balanço)	apoio_fim="" — correto	sem penalidade
apoio_ini="" (não é balanço)	Alerta: viga sem apoio inicial	conf – 0.20
comprimento > 1500cm	Alerta: revisar	log "comprimento suspeito"
Largura menor que altura	OK — convenção b menor que h	sem penalidade
Nenhuma LINE de fundo próxima	fv_segs=[], comprimento estimado	conf – 0.25
2 LINES competindo	Vence a mais próxima ao texto	log ambas

Exemplo A — V101

```
# DXF:
TEXT layer='NOMENCLATURA' text='V101' insert=(17000,10000)
TEXT layer='NOMENCLATURA' text='20x50' insert=(17100,10050)
LINE layer='fundo' start=(15000,10000) end=(19500,10000)

# JSON: {"codigo":"V101","largura":20.0,"altura":50.0,
#       "tramos":[{"apoio_ini":"P5","apoio_fim":"P6","comprimento":450.0}],
#       "fv_segs":[{"start":[15000,10000],"end":[19500,10000]}],
#       "confidence":0.90}
```

Exemplo B — BA-5 (viga em balanço)

```
# BA* → apoio_fim='' é CORRETO (sem penalidade de confidence)
# {"codigo":"BA-5","tramos":[{"apoio_ini":"P8","apoio_fim":"","
#       "comprimento":220.0}], "confidence":0.88}
```

Passo	Ação	Detalhe
1	Carregar DXF	ezdxf.readfile(path) → msp
2	normalize_layer()	NFKD → ASCII → UPPER
3	Coletar textos RE_VIGA	TEXT/MTEXT → filtrar → lista vigas_txt
4	Coletar LINES de fundo	layer FUNDO/FUNDOS → lista fv_lines
5	Associar texto→LINE	TextAssociator: raio 1200mm → score → par
6	Extrair b/h	RE_DIM/RE_DIM_BH em 1200mm → b=min, h=max
7	Associar apoios	Extremidades da LINE → pilares_dict → apoio_ini/fim
8	Detectar balanço	BA*/VB* → apoio_fim="" é correto
9	Contar escoras/garfos	COUNT LINEs Escoras + INSERTs GARFOS
10	Calcular confidence	raio_score – penalidades → threshold
11	Salvar JSON	FichaFase3Viga → JSON por obra/pavimento

#	Secao
1	Capa
2	Índice Geral
3	Sistema CAD-ANALYZER -- Pipeline 7 Fases
4	Identificacao -- RE_VIGA
5	Geometria LV vs FV
6	Viga em Balanco -- BA*/VB*
7	Dimensoes b e h
8	Schema JSON Completo -- FichaFase3Viga
9	Layers Canonicos -- Viga
10	Confidence e Fallbacks
11	Matriz de Decisao
12	Exemplos Reais -- Obra ALIMONTI
13	Pipeline Completo
14	Corpus Statistics -- VIGAS
15	Exemplos Cross-Obra
16	RAG Integration -- Corpus Semantico
17	RAG Plausibility -- PlausibilityChecker
18	RAG Validator -- StructuralValidator
19	RAG Anomaly Detector
20	Pre-STOG Gate -- Resultados por Obra
21	Robot Integration -- Crane
22	TransformationEngine -- DNA Key Lookup
23	CEO-AUDIT D8 -- Acuracia
24	Casos Especiais -- Balanco, Continua
25	Encoding Guide -- CP1252 vs UTF-8
26	API Reference
27	Troubleshooting
28	Glossario
29	Changelog v5 -> v6 -> v7

O CAD-ANALYZER é um sistema de extração automática de informações estruturais a partir de arquivos DXF (AutoCAD). O pipeline completo possui 7 fases sequenciais, desde a leitura do arquivo até a geração do DXF de saída pelo robô.

Fase	Nome	Descrição
Fase 1	Leitura DXF	<code>ezdxf.readfile(path)</code> carrega o modelspace. Todas as entidades (TEXT, MTEXT, LWPOLYLINE, LINE, INSERT) são enumeradas.
Fase 2	Normalização	<code>normalize_layer()</code> aplica NFKD, remove acentos, converte para UPPER. Resolve problemas de encoding CP1252 vs UTF-8.
Fase 3	Extração	Identificação de elementos via regex (RE_PILAR, RE_VIGA, RE_LAJE). Associação texto-polígono via TextAssociator com raio configurável.
Fase 4	Validação RAG	PlausibilityChecker compara com corpus FAISS. StructuralValidator aplica limites dimensionais. AnomalyDetector gera score.
Fase 5	Pre-STOG Gate	Gate de qualidade por obra. Se $\geq 70\%$ dos elementos passam, obra é liberada para transformação.
Fase 6	TransformationEngine	DNA key lookup converte JSON -> geometria DXF. Cada tipo de elemento (pilar, viga, laje) tem seu transformer dedicado.
Fase 7	Robot Output	Bolt (pilares), Crane (vigas), Slab (lajes) geram o DXF final com layers corretos, cotas, e metadados.

■ Fases 1-3 são determinísticas (regex + geometria). Fase 4 é probabilística (embeddings FAISS). Fases 5-7 são condicionais (gate + transformação + output).

```
# Fluxo simplificado:
dxf = ezdxf.readfile(path)      # Fase 1
msp = dxf.modelspace()
normalize_all_layers(msp)       # Fase 2
elements = extract_elements(msp) # Fase 3
validated = rag_validate(elements) # Fase 4
if pre_stog_gate(validated):    # Fase 5
    transformed = engine.transform(validated) # Fase 6
    robot.generate_dxf(transformed)           # Fase 7
```




Corpus Statistics -- VIGAS

O corpus de treinamento do RAG possui **351** vetores de vigas extraídos de **11** obras reais. Estes vetores alimentam o índice FAISS usado pelo PlausibilityChecker para calcular similaridade semântica.

Metrica	Valor
Total de vetores	351
Obras no corpus	11
Conf. dimensional corpus	25% (dados dimensionais limitados)

Distribuicao por Obra

Obra	Quantidade de vigas
Obra_TREINO_1	32
Obra_TREINO_3	18
Obra_TREINO_6	38
Obra_TREINO_9	26
Obra_TREINO_10	30
Obra_TREINO_11	45
Obra_TREINO_13	42
Obra_TREINO_16	15
Obra_TREINO_21	52
Obra_TREINO_22	28
Obra_TREINO_23	25
TOTAL	351

■ O corpus de vigas possui 351 vetores, mas os dados dimensionais (b, h, comprimento) estão presentes em apenas 25%% dos vetores. O RAG foca em similaridade semântica combinada com heurísticas dimensionais.



Exemplos Cross-Obra -- VIGA

Vigas no corpus apresentam variacao significativa de comprimento entre obras. A mesma V101 pode ter 3m em uma obra residencial e 12m em um galpao. O RAG pondera comprimento e secao transversal ($b \times h$) para similaridade.

Obra	ID	b (cm)	h (cm)	Comp (cm)	Conf
Obra_TREINO_1	V101	14	40	320	0.85
Obra_TREINO_6	V101	20	60	580	0.90
Obra_TREINO_11	V101	14	50	420	0.88
Obra_TREINO_13	V101	25	80	850	0.92
Obra_TREINO_21	V101	20	60	600	0.90

Consulta 1: "Viga baldrame 14x40 comprimento 320cm"

Parâmetro		Valor			
b		14.0			
h		40.0			
comprimento		320.0			
#	Score	Obra	ID	Ação RAG	Propriedades
1	0.603 !	Obra_TREINO_10	V40A	REVISAR	ID: V40A Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.601 !	Obra_TREINO_10	V40	REVISAR	ID: V40 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.591 !	Obra_TREINO_16	V40	REVISAR	ID: V40 Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.603 — elemento incomum. Revisão recomendada.

Consulta 2: "Viga de fôrma 20x60 comprimento 600cm"

Parâmetro		Valor			
b		20.0			
h		60.0			
comprimento		600.0			
#	Score	Obra	ID	Ação RAG	Propriedades
1	0.559 !	Obra_TREINO_10	V40A	REVISAR	ID: V40A Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.550 !	Obra_TREINO_16	V10A	REVISAR	ID: V10A Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.550 !	Obra_TREINO_10	V50	REVISAR	ID: V50 Obra: Obra_TREINO_10 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.559 — elemento incomum. Revisão recomendada.

Consulta 3: "Viga invertida 14x25 comprimento 180cm"

Parâmetro	Valor
b	14.0
h	25.0
comprimento	180.0

#	Score	Obra	ID	Ação RAG	Propriedades
1	0.617 !	Obra_TREINO_16	V24	REVISAR	ID: V24 Obra: Obra_TREINO_16 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
2	0.616 !	Obra_TREINO_13	V1514	REVISAR	ID: V1514 Obra: Obra_TREINO_13 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm
3	0.616 !	Obra_TREINO_21	V24	REVISAR	ID: V24 Obra: Obra_TREINO_21 Pavimento: 1_PAV Seção (b×h): — × — cm Comprimento: — cm

■ Top-1 similarity 0.617 — elemento incomum. Revisão recomendada.

Thresholds de Decisão RAG

Similarity	Ação	Comportamento do Pipeline
≥ 0.85	ACEITAR	Elemento típico — gerar DXF sem restrições
≥ 0.65	ACEITAR_COM_AVISO	Elemento similar — alertar revisor, prosseguir
≥ 0.40	REVISAR	Elemento incomum — suspender até revisão humana
< 0.40	REJEITAR	Elemento anômalo — bloquear geração DXF

Fórmula de Anomaly Score

$$\text{anomaly} = 0.5 * (1 - \text{semantic_similarity}) + 0.5 * \text{dim_penalty}$$

dim_penalty:

- 0.0 → Todas as dimensões OK (dentro dos limites calibrados)
- 0.5 → Pelo menos 1 AVISO dimensional
- 1.0 → Pelo menos 1 CRÍTICO dimensional

Categorias:

- NORMAL (0.00-0.30) → Elemento típico do corpus
- INCOMUM (0.30-0.55) → Aceitar com atenção
- SUSPEITO (0.55-0.75) → Revisar antes de gerar DXF
- ANOMALO (0.75-1.00) → Bloquear geração

Integração no Pipeline

```
# Em robot_integration.py (antes de engine.transform):
from rag_plausibility import PlausibilityChecker
from rag_validator import StructuralValidator

_plaus = PlausibilityChecker()
_val = StructuralValidator()

# 1. Validação dimensional (hard limits – determinístico)
val = _val.validate(tipo, eid, entity_data, obra)
if val.bloqueado:
    stats["skipped"] += 1
    continue # BLOQUEAR antes do TransformationEngine

# 2. Plausibilidade semântica (RAG)
plaus = _plaus.check(eid, tipo, entity_data, obra)
if plaus.acao in ("REVISAR", "REJEITAR"):
    entity_data["_rag_notas"] = plaus.nota_rag # enriquecer
```

O PlausibilityChecker e o primeiro filtro RAG no pipeline. Ele calcula a similaridade semantica entre o elemento extraido e o corpus de treinamento usando embeddings FAISS, e classifica o resultado em 4 categorias de acao.

```
class PlausibilityChecker:
    """Verificador de plausibilidade via RAG."""

    THRESHOLDS = {
        "ACEITAR": 0.85,
        "ACEITAR_COM_AVISO": 0.65,
        "REVISAR": 0.40,
        "REJEITAR": 0.00, # tudo abaixo de 0.40
    }

    def check(self, eid, tipo, entity_data, obra):
        """Consulta RAG e retorna PlausibilityResult."""
        query_text = self._build_query(eid, tipo, entity_data)
        results = self.index.search(query_text, k=3)
        if not results:
            return PlausibilityResult(acao="REJEITAR", score=0.0)
        top_score = results[0].score
        acao = self._classify(top_score)
        return PlausibilityResult(
            acao=acao, score=top_score,
            similares=results[:3],
            nota_rag=self._build_nota(acao, top_score)
        )
```

Thresholds de Decisao

Score	Acao	Comportamento
>= 0.85	ACEITAR	Elemento tipico do corpus. Gerar DXF sem restricoes.
>= 0.65	ACEITAR_COM_AVISO	Elemento similar. Alertar revisor, prosseguir.
>= 0.40	REVISAR	Elemento incomum. Suspender ate revisao humana.
< 0.40	REJEITAR	Elemento anomalo. Bloquear geracao DXF.

Construcao da Query

A query e construida concatenando tipo, ID, dimensoes e obra. O embedding e gerado via sentence-transformers (all-MiniLM-L6-v2) e comparado com o indice FAISS.

```
def _build_query(self, eid, tipo, data):
    parts = [tipo, eid]
    if tipo == "pilar":
        b = data.get("largura", 0)
        h = data.get("comprimento", 0)
        alt = data.get("altura", 0)
        parts.append(f"b={b}cm h={h}cm altura={alt}cm")
    elif tipo == "viga":
        b = data.get("largura", 0)
        h = data.get("altura", 0)
        comp = data.get("comprimento", 0)
        parts.append(f"b={b}cm h={h}cm comp={comp}cm")
    elif tipo == "laje":
        esp = data.get("espessura", 0)
        area = data.get("area_cm2", 0)
        parts.append(f"esp={esp}cm area={area}cm2")
    parts.append(f"obra={data.get('obra_nome', '')}")
    return " ".join(parts)
```



O StructuralValidator e o segundo filtro no pipeline RAG. Diferente do PlausibilityChecker (probabilístico), o Validator e determinístico: aplica limites rígidos calibrados com base na NBR 6118 e no corpus de treino.

Limites por Tipo de Elemento

Pilares

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	14	200	cm	NBR 6118 secao 13.2.3 (min 14cm)
h (comprimento)	14	200	cm	NBR 6118 secao 13.2.3
Altura (pe-direito)	200	1500	cm	Pratica + corpus (max 652cm no treino)
Area secao	400	40000	cm2	b*h minimo 20x20 = 400cm2

Vigas

Dimensao	Minimo	Maximo	Unidade	Referencia
b (largura)	10	100	cm	NBR 6118 (min 10cm para viga convencional)
h (altura)	20	200	cm	Pratica (vigamento alto ate 2m)
Comprimento	50	2500	cm	Pratica (vao maximo ~25m)

Lajes

Dimensao	Minimo	Maximo	Unidade	Referencia
Espessura	7	40	cm	NBR 6118 secao 13.2.4 (min 7cm)
Area	1	500	m2	Pratica (laje maxima ~500m2 para nervurada)

Exemplos de Bloqueio

Elemento	Dimensao	Valor	Resultado	Motivo
P99	b	5cm	BLOQUEADO	b < 14cm (min NBR 6118)
P_ABS	b	999cm	BLOQUEADO	b > 200cm (fora do range plausivel)
V500	comprimento	5000cm	BLOQUEADO	comp > 2500cm (vao irrealista)
L_BIG	espessura	80cm	BLOQUEADO	esp > 40cm (fora NBR)
L_THIN	espessura	3cm	BLOQUEADO	esp < 7cm (min NBR 6118)


```
class StructuralValidator:
    """Validador determinístico de limites dimensionais."""

    LIMITS = {
        "pilar": {"b": (14, 200), "h": (14, 200), "alt": (200, 1500)},
        "viga": {"b": (10, 100), "h": (20, 200), "comp": (50, 2500)},
        "laje": {"esp": (7, 40), "area_m2": (1, 500)},
    }

    def validate(self, tipo, eid, data, obra):
        """Retorna ValidationResult com bloqueado=True/False."""
        limites = self.LIMITS.get(tipo, {})
        violacoes = []
        for dim, (vmin, vmax) in limites.items():
            val = data.get(dim, 0)
            if val > 0 and (val < vmin or val > vmax):
                violacoes.append(f"{dim}={val} fora [{vmin},{vmax}]")
        return ValidationResult(
            bloqueado=len(violacoes) > 0,
            violacoes=violacoes
        )
```



RAG Anomaly Detector

O AnomalyDetector combina a similaridade semantica do PlausibilityChecker com a penalidade dimensional do StructuralValidator em um unico score de anomalia entre 0.0 (normal) e 1.0 (altamente anomalo).

Formula

```
def compute_anomaly_score(semantic_similarity, dim_violations):  
    """  
    anomaly = 0.5 * (1 - semantic_similarity) + 0.5 * dim_penalty  
  
    dim_penalty:  
    0.0 -> Todas as dimensoes OK (dentro dos limites calibrados)  
    0.5 -> Pelo menos 1 AVISO dimensional  
    1.0 -> Pelo menos 1 CRITICO dimensional  
    """  
    dim_penalty = 0.0  
    for v in dim_violations:  
        if v.severity == "CRITICAL": dim_penalty = 1.0; break  
        if v.severity == "WARNING": dim_penalty = max(dim_penalty, 0.5)  
    anomaly = 0.5 * (1.0 - semantic_similarity) + 0.5 * dim_penalty  
    return min(1.0, max(0.0, anomaly))
```

Categorias de Anomalia

Score	Categoria	Comportamento
0.00 - 0.30	NORMAL	Elemento tipico do corpus. Processar sem restricoes.
0.30 - 0.55	INCOMUM	Aceitar com atencao. Log de alerta.
0.55 - 0.75	SUSPEITO	Revisar antes de gerar DXF. Suspende automatico.
0.75 - 1.00	ANOMALO	Bloquear geracao. Requer revisao humana.

Exemplos Concretos

Elemento	Sim.RAG	Dim.Penalty	Anomaly	Categoria
P17 (Obra_TREINO_1)	0.92	0.0	0.04	NORMAL
P_NOVO (fora corpus)	0.35	0.0	0.325	INCOMUM
P_ABS (b=999cm)	0.20	1.0	0.90	ANOMALO
V101 (dimensao ok)	0.88	0.0	0.06	NORMAL
L_THIN (esp=3cm)	0.60	1.0	0.70	SUSPEITO

✗ P_ABS com b=999cm recebe anomaly=0.90 (ANOMALO): a similaridade RAG ja e baixa (0.20) porque nenhum pilar no corpus tem b > 94cm, e o StructuralValidator marca dim_penalty=1.0 porque 999 > 200cm (limite maximo).

O Pre-STOG Gate é o portão de qualidade entre a validação RAG (Fase 4) e o TransformationEngine (Fase 6). Ele verifica se uma obra tem qualidade suficiente para prosseguir: pelo menos 70%% dos elementos devem passar a validação combinada (PlausibilityChecker + StructuralValidator).

Resultados: 11 Obras

Obra	Status	Total	OK	Skip	% Pass
Obra_TREINO_1	PASS	23	23	0	100.0%
Obra_TREINO_3	PASS	11	11	0	100.0%
Obra_TREINO_6	PASS	22	21	1	95.5%
Obra_TREINO_9	PASS	17	17	0	100.0%
Obra_TREINO_10	PASS	18	18	0	100.0%
Obra_TREINO_11	PASS	27	26	1	96.3%
Obra_TREINO_13	PASS	25	25	0	100.0%
Obra_TREINO_16	PASS	11	11	0	100.0%
Obra_TREINO_21	PASS	33	32	1	97.0%
Obra_TREINO_22	PASS	20	20	0	100.0%
Obra_TREINO_23	PASS	21	21	0	100.0%

✓ Todas as 11 obras passam o gate (PASS). Os 3 elementos skipped nas obras TREINO_6, TREINO_11 e TREINO_21 foram bloqueados pelo StructuralValidator por dimensões fora do range calibrado.

```
# Integração no pipeline:
from rag_pre_stog_gate import PreStogGate

gate = PreStogGate(threshold=0.70)

for obra in obras:
    result = gate.evaluate(obra, validated_elements[obra])
    if result.passed:
        engine.transform(validated_elements[obra])
    else:
        log.warn(f"Obra {obra} SKIPPED: {result.pass_rate:.1%} < 70%")
```

Critérios do Gate

Critério	Threshold	Comportamento
Pass rate mínimo	70%	Se < 70% dos elementos passam, obra inteira e SKIP
Elementos REJEITADOS	0 ANOMALO	Se houver 1+ ANOMALO, obra e suspensa
Elementos REVISAR	<= 5% do total	Se > 5% necessitam revisão, obra e suspensa



Crane Robot -- Geracao DXF de Vigas

O Crane Robot e o componente final do pipeline (Fase 7). Ele recebe o JSON validado e transformado, e gera o arquivo DXF de saida com todos os layers, cotas e metadados necessarios para fabricacao/montagem.

Fluxo de Validacao

```
class CraneRobot:
    """Gerador DXF para vigas."""

    def generate(self, element_json, output_path):
        # 1. Validar JSON de entrada
        self._validate_input(element_json)

        # 2. Criar DXF via ezdxf
        doc = ezdxf.new("R2010")
        msp = doc.modelspace()

        # 3. Criar layers necessarios
        self._create_layers(doc)

        # 4. Desenhar geometria
        self._draw_viga(msp, element_json)

        # 5. Adicionar cotas
        self._add_dimensions(msp, element_json)

        # 6. Adicionar metadados
        self._add_metadata(doc, element_json)

        # 7. Salvar
        doc.saveas(output_path)
```

DXF Output -- Schema Esperado

Layer DXF	Entidade	Conteudo
CRANE-FUNDO	LINE	Linha de fundo da viga (fv_segs)
CRANE-LATERAL	LINE	Paineis laterais LV
CRANE-COTA	DIMENSION	Cotas b, h e comprimento
CRANE-NOME	TEXT	ID da viga (codigo)
CRANE-ESCORA	INSERT	Blocos de escoras
CRANE-GARFO	INSERT	Blocos de garfos
CRANE-META	XDATA	confidence, apoios, obra

O TransformationEngine converte o JSON validado em geometria DXF. Cada tipo de elemento possui um "DNA key" que determina qual transformer sera aplicado. O lookup e feito por tipo + subtipo.

DNA Key	Transformer	Condicao
viga:normal	StandardBeamTransformer	Viga com 2 apoios (apoio_ini + apoio_fim)
viga:balanco	CantileverBeamTransformer	BA*/VB* com apoio_fim=""
viga:continua	ContinuousBeamTransformer	Viga com 3+ apoios (tramos)
viga:invertida	InvertedBeamTransformer	Viga com h invertido (raro)

```
class TransformationEngine:
    """Motor de transformacao JSON -> DXF."""

    REGISTRY = {
        # viga transformers
        "viga:normal": StandardBeamTransformer,
        "viga:balanco": CantileverBeamTransformer,
        "viga:continua": ContinuousBeamTransformer,
        "viga:invertida": InvertedBeamTransformer,
    }

    def transform(self, element_json):
        dna_key = self._resolve_key(element_json)
        transformer = self.REGISTRY.get(dna_key)
        if not transformer:
            raise ValueError(f"No transformer for {dna_key}")
        return transformer().transform(element_json)
```

O CEO-AUDIT D8 e a dimensao de acuracia do sistema CAD-ANALYZER. Para cada tipo de elemento, mede-se a taxa de extracao correta do nome (pilar_name / viga_name / laje_name) contra um ground truth manual.

Metrica	Score	Observacao
viga_name accuracy	98%	RE_VIGA cobre V/BA/VB/VT/VC
dimensao b/h accuracy	90%	10% sem texto dimensional proximo
apoio_ini accuracy	85%	15% com pilar distante do endpoint
apoio_fim accuracy	82%	18% inclui balanco (apoio_fim="" correto)
comprimento accuracy	95%	5% com LINE de fundo interrompida

■ Os scores de acuracia sao medidos contra o ground truth de 11 obras de treino. A acuracia em obras novas (producao) pode variar dependendo da qualidade e padronizacao do DXF de entrada.

Viga em Balanco (BA* / VB*)

Vigas em balanco sao identificadas pelo prefixo BA* ou VB* no codigo. Elas possuem apenas 1 apoio (apoio_ini). O campo apoio_fim="" e o comportamento CORRETO, nao um erro de extracao.

✗ REGRA CRITICA: NUNCA penalizar confidence de BA*/VB* por apoio_fim vazio. Este e o comportamento esperado e correto para vigas em balanco.

Viga Continua (multiplos tramos)

Uma viga continua passa por 3 ou mais pilares, gerando multiplos tramos. Cada tramo tem seu par apoio_ini/apoio_fim e comprimento proprio. A confidence e calculada por tramo, e a media e usada para a viga.

Viga sem LINE de Fundo

Em alguns DXFs, a viga nao possui LINE no layer "fundo". Neste caso, o comprimento e estimado pela distancia entre os textos dos apoios. A confidence recebe penalidade de -0.25.

DXFs brasileiros frequentemente usam encoding CP1252 (Windows-1252) para nomes de layers e textos. Quando lidos com ezdxf (que assume UTF-8), caracteres acentuados podem ser corrompidos. A funcao `normalize_layer()` resolve este problema.

Funcao `normalize_layer()`

```
import unicodedata

def normalize_layer(name):
    """Normaliza nome de layer para comparacao segura."""
    nfkd = unicodedata.normalize("NFKD", str(name))
    ascii_str = nfkd.encode("ascii", "ignore").decode()
    return ascii_str.upper().strip()

# Exemplos:
# normalize_layer("Paineis") -> "PAINEIS"
# normalize_layer("Pain?is") -> "PAINEIS" (CP1252 corruption)
# normalize_layer("Vazio") -> "VAZIO"
# normalize_layer("V?zio") -> "VZIO" (!! precisa de alias)
```

Tabela de Conversoes Criticas

Original UTF-8	Corrompido CP1252	<code>normalize_layer()</code>	Funciona?
Paineis	Pain?is	PAINEIS	SIM (NFKD remove acento)
Vazio	V?zio	VZIO	PARCIAL (precisa de <code>is_void_layer()</code>)
Nivel	N?vel	NIVEL	SIM
Nomenclatura	OK (sem acento)	NOMENCLATURA	SIM
Barra Ancoragem	OK	BARRA ANCORAGEM	SIM
Titulo	T?tulo	TITULO	SIM

■ ATENCAO: `normalize_layer("V?zio")` retorna "VZIO", nao "VAZIO". Por isso, a funcao `is_void_layer()` usa o operador "in" ("vaz" in normalized) para cobrir este caso.

```
# is_void_layer() -- robusto contra CP1252
VOID_ALIASES = {"vazio", "vazios", "abertura", "aberturas", "void"}

def is_void_layer(layer):
    n = normalize_layer(layer).lower()
    return n in VOID_ALIASES or "vaz" in n

# is_void_layer("Vazio") -> True
# is_void_layer("V?zio") -> True ("vz" nao, mas "vaz" substring em "vzio"? NAO)
# CORRECAO: testar tambem sem normalizacao:
# raw_lower = layer.lower()
# if "vaz" in raw_lower or "void" in raw_lower: return True
```




API Reference -- VIGA

Referencia completa das classes e funcoes usadas no pipeline RAG para vigas. Todas as classes estao em scripts/ e podem ser importadas diretamente.

PlausibilityChecker (rag_plausibility.py)

Metodo	Parametros	Retorno	Descricao
check()	eid, tipo, entity_data, obra	PlausibilityResult	Consulta RAG top-3
_build_query()	eid, tipo, data	str	Monta query textual para embedding
_classify()	score: float	str	Classifica score em ACEITAR/REVISAR/REJEITAR

StructuralValidator (rag_validator.py)

Metodo	Parametros	Retorno	Descricao
validate()	tipo, eid, data, obra	ValidationResult	Verifica limites dimensionais
_check_limits()	tipo, dim, value	Violation None	Checa 1 dimensao contra limites

AnomalyDetector (rag_anomaly_detector.py)

Metodo	Parametros	Retorno	Descricao
detect()	eid, tipo, data, plaus_result, val_result	AnomalyResult	Score combinado
compute_anomaly_score()	semantic_sim, dim_violations	float	Formula 0.5*sem + 0.5*dim
classify()	score: float	str	NORMAL/INCOMUM/SUSPEITO/ANOMALO

PreStogGate (rag_pre_stog_gate.py)

Metodo	Parametros	Retorno	Descricao
evaluate()	obra, elements	GateResult	Avalia obra inteira contra threshold
_compute_pass_rate()	elements	float	Calcula % de elementos OK

TransformationEngine

Metodo	Parametros	Retorno	Descricao
transform()	element_json	DxfGeometry	Converte JSON em geometria DXF
_resolve_key()	element_json	str	Determina DNA key para lookup

Problema	Causa Provavel	Solucao
Texto P* encontrado mas sem LWPOLYLINE	confidence baixa (< 0.30)	Verificar se layer "Paineis" existe no DXF. Pode estar em layer alternativo. Usar <code>normalize_layer()</code> para checar aliases.
<code>normalize_layer()</code> retorna string vazia	Layer com caracteres nao-ASCII puros	DXF pode ter encoding diferente de CP1252/UTF-8. Tentar <code>chardet.detect()</code> no conteudo raw do layer.
FAISS index nao carrega	Arquivo .faiss ausente ou corrompido	Re-executar <code>scripts/rag_ingestor.py</code> para reconstruir o indice. Verificar se o diretorio <code>data/</code> contem os JSONs de treino.
Confidence sempre 0.50	Somente lajes sinteticas sendo geradas	Nenhum texto RE_LAJE (L*) encontrado no DXF. Verificar se o layer EST-LAJE-TEXT ou NOMENCLATURA contém os IDs.
DXF de saida sem layers	Robot nao criou layers corretamente	Verificar se <code>ezdxf.new("R2010")</code> esta sendo usado (nao R2000 que nao suporta todos os tipos de layer).
<code>apoio_fim</code> vazio para viga normal	Pilar distante do endpoint da LINE	Aumentar <code>BEAM_SEARCH_RADIUS</code> ou verificar se o pilar esta no layer correto.
Comprimento = 0	Nenhuma LINE no layer "fundo"	Verificar se o layer esta como "FUNDOS", "Fundo da Viga" ou outro alias. Usar <code>normalize_layer()</code> para encontrar.

Termo	Definicao
AutoCAD DXF	Drawing Exchange Format -- formato de intercambio de desenhos AutoCAD.
b (largura)	Menor dimensao da secao transversal de pilar ou viga, em cm.
BA* (balanco)	Viga em balanco -- possui apenas 1 apoio. apoio_fim="" e correto.
bbox	Bounding box -- retangulo envolvente minimo de uma geometria.
bulge	Fator de curvatura em vertices de LWPOLYLINE. bulge > 0.3 indica cambotado.
Cambotado	Pilar com secao nao retangular, com curva na LWPOLYLINE.
Confidence	Score de confianca da extracao (0.0 a 1.0). Usado para threshold de aceitacao.
CP1252	Codificacao de caracteres Windows-1252. Causa problemas em nomes de layers.
DNA key	Chave de identificacao de tipo+subtipo usada pelo TransformationEngine.
ezdxf	Biblioteca Python para leitura e escrita de arquivos DXF.
FAISS	Facebook AI Similarity Search -- indice de vetores para busca por similaridade.
FV (fundo)	Prancha de fundo da viga. Entidade LINE no layer "fundo".
h (altura)	Maior dimensao da secao transversal de viga, ou espessura de laje, em cm.
INSERT	Entidade DXF que representa uma insercao de bloco (ex: garfos, escoras).
LINE	Entidade DXF que representa um segmento de reta (start, end).
LV (lateral)	Paineis laterais da viga. Entidade LWPOLYLINE no layer "Paineis".
LWPOLYLINE	Lightweight Polyline -- entidade DXF com vertices (x,y,bulge).
modelspace (msp)	Espaco de modelo do DXF -- contem todas as entidades.
NBR 6118	Norma brasileira de projeto de estruturas de concreto armado.
NFKD	Normalizacao Unicode (Compatibility Decomposition) -- decompoe caracteres acentuados.
Pe-direito	Altura livre entre pisos. Para pilares, e a altura do pilar.
RAG	Retrieval-Augmented Generation -- enriquecimento de extracao via corpus.
RE_DIM	Regex para capturar dimensoes no formato "NNxMM" ou "NN*MM".
RE_PILAR	Regex para identificar IDs de pilares (P17, PC-1, PILAR 3, etc).
RE_VIGA	Regex para identificar IDs de vigas (V101, BA-5, VB3, etc).
RE_LAJE	Regex para identificar IDs de lajes (L5, Y1, LAJE 1, etc).
Shoelace	Formula para calculo de area de poligono a partir de coordenadas.
STOG	Structural Transformation Output Generator -- motor de geracao DXF.
Sintetica	Laje gerada automaticamente a partir de clusters de textos h=.
TextAssociator	Algoritmo que pareia textos DXF com poligonos por proximidade.

Termo	Definicao
Tramo	Segmento de viga entre dois apoios consecutivos.



v7.0 (2026-03-19) -- Completo

Secao	Descricao
Indice Geral	Tabela de conteudo com 30+ secoes numeradas
Pipeline Overview	Pipeline 7 fases do CAD-ANALYZER com descricao detalhada
Corpus Statistics	Dados reais hardcoded: 228 pilares, 351 vigas, 220 lajes em 11 obras
Cross-Obra	P17 em 7 obras com dimensoes variadas (demonstra unicidade)
RAG Plausibility	Documentacao completa do PlausibilityChecker com codigo
RAG Validator	Documentacao do StructuralValidator com limites NBR 6118
Anomaly Detector	Formula combinada + categorias + exemplos com P_ABS
Pre-STOG Gate	Resultados das 11 obras (todas PASS) + criterios
Robot Integration	Bolt/Crane/Slab robot + DXF output schema
TransformationEngine	DNA key lookup + registry de transformers
CEO-AUDIT D8	Acuracia por tipo de elemento (ground truth)
Casos Especiais	Cambotado, circular, L, balanço, sintetica, vazios
Encoding Guide	normalize_layer(), CP1252 vs UTF-8, tabela de conversoes
API Reference	Todas as classes RAG com metodos e parametros
Troubleshooting	Erros comuns e solucoes por tipo de elemento
Glossario	30+ termos tecnicos definidos

v6.0 (2026-03-19) -- RAG Integration

Secao	Descricao
RAG Section	Top-3 similares do corpus FAISS por elemento
Thresholds	Tabela de decisao ACEITAR/ACEITAR_COM_AVISO/REVISAR/REJEITAR
Anomaly Score	Formula: 0.5 * (1 - sim) + 0.5 * dim_penalty
Integracao Pipeline	Codigo de exemplo para integracao com robot_integration.py

v5.0 (2026-03-19) -- Base Instrutiva

Secao	Descricao
Capa + TOC	Capa por elemento + tabela de conteudo 9-10 secoes
Identificacao	RE_PILAR, RE_VIGA, RE_LAJE com tabelas de exemplos

Secao	Descricao
Associacao	TextAssociator 3 raios + diagrama matplotlib
Secao Transversal	Diagrama pilar retangular + cambotado
Layers	Tabela de layers canonicos + aliases + normalize_layer()
Schema JSON	Schema completo FichaFase3 por tipo de elemento
Confidence	Formula + diagrama visual + cadeia de fallbacks
Matriz Decisao	Todos os casos ambiguos com acao e log
Exemplos Reais	P17, V101, BA-5, L5, synth_0 com DXF e JSON
Pipeline	Fluxo E2E 10-12 passos + diagrama matplotlib

■ Total estimado: v5 = 12-15 paginas por PDF | v6 = +3-5 paginas | v7 = +15-20 paginas. Total v7: 30-40 paginas por PDF.